

Carga e Transformação de Dados

Modelo Analítico de Dados para E-commerce

Sumário

1	Introdução	3
2	Preparação Automatizada do Banco	3
2.1	Identificação do banco de destino	3
2.2	Criação automática do database	4
2.3	Aplicação do modelo físico	4
2.4	Reconstrução controlada	4
3	Arquitetura do ELT	5
3.1	Camada RAW	5
3.2	Camada CORE	5
3.3	Atomicidade do pipeline completo	6
4	Contrato de Ingestão RAW	6
5	Mapeamento RAW para CORE	7
6	Regras de Qualidade e Validação	8
7	Implementação	9
8	Configuração e Execução	10
9	Validação e Testes	12
10	Resultado da Carga	13
10.1	Resultado da validação pós-carga	14
11	Rastreabilidade e Limites	14
12	Conclusão	15

1 Introdução

Este documento consolida a preparação automatizada do banco de dados, o processo de carga, a transformação e a validação dos dados do projeto *Modelo Analítico de Dados para E-commerce*, desde a criação da estrutura física no PostgreSQL até a publicação e auditoria das entidades relacionais no schema `core`.

A preparação da infraestrutura de banco é tratada separadamente do ELT. O componente `database` garante que o banco de destino exista, aplica os scripts versionados do modelo físico, cria os índices e valida a arquitetura. Somente após essa etapa o pipeline ELT realiza a carga dos nove arquivos CSV do *Brazilian E-Commerce Public Dataset by Olist* na RAW e a transformação para o CORE.

A validação pós-carga é executada de forma independente pelo componente `validation`. Seu objetivo é confirmar volumes, integridade, qualidade, nulabilidade e correspondência entre os valores esperados a partir da RAW e os dados efetivamente publicados no CORE.

A solução busca garantir reprodutibilidade, rastreabilidade, integridade, atomicidade, reconciliação entre fonte e destino e validação independente do resultado publicado.

PostgreSQL → database.setup → RAW → CORE → validation → ANALYTICS → Power BI / SQL

As decisões são descritas por sua finalidade técnica e semântica, sem depender de identificadores internos utilizados no gerenciamento do projeto.

2 Preparação Automatizada do Banco

A criação e a preparação da estrutura física são responsabilidades anteriores ao ELT e, por esse motivo, permanecem isoladas no pacote `database`. Os scripts SQL em `models/physical/` continuam sendo a fonte da verdade para o modelo físico; o código Python atua apenas como orquestrador.

2.1 Identificação do banco de destino

A conexão é definida pela variável de ambiente `DATABASE_URL`. O nome do banco é extraído da própria URL e registrado no log antes de qualquer alteração estrutural.

No ambiente local, a configuração pode ser mantida em arquivo `.env`, ignorado pelo Git:

```
DATABASE_URL=postgresql://usuario:senha@localhost:5432/ecommerce_analytics
```

O arquivo `.env.example` documenta a variável necessária sem expor credenciais reais.

2.2 Criação automática do database

Antes de aplicar a estrutura física, o processo consulta o catálogo `pg_database` por meio do banco administrativo `postgres`. Se o database indicado em `DATABASE_URL` ainda não existir, ele é criado automaticamente.

Essa operação depende de o usuário PostgreSQL possuir permissão para `CREATE DATABASE`. Caso a permissão não esteja disponível, a execução é interrompida e o erro é informado sem tentativa de contorno.

A verificação por catálogo evita dependência de mensagens textuais do servidor e, portanto, não depende do idioma configurado no PostgreSQL.

2.3 Aplicação do modelo físico

Após garantir a existência do database, o processo aplica os artefatos físicos versionados nesta ordem:

1. `models/physical/create_schema.sql`;
2. `models/physical/create_indexes.sql`;
3. `models/physical/validate_schema.sql`.

A validação final confirma a presença dos schemas, tabelas, colunas, identities, PKs, FKs, constraints e índices definidos para a arquitetura física.

2.4 Reconstrução controlada

Quando o banco já existe, a opção `-reset` executa `drop_schema.sql` antes da recriação da estrutura:

```
uv run python -m database.setup --reset
```

O `-reset` não remove o database PostgreSQL. Apenas a estrutura pertencente ao projeto é reconstruída. Se o database for criado na própria execução, a etapa de remoção é omitida por não existir estrutura anterior a ser descartada.

Para um ambiente novo, a preparação pode ser executada simplesmente por:

```
uv run python -m database.setup
```

3 Arquitetura do ELT

3.1 Camada RAW

A camada RAW preserva os valores recebidos da fonte. Existe uma tabela correspondente a cada CSV; as colunas de negócio mantêm os nomes originais e são armazenadas inicialmente como `text`.

Cada ocorrência recebe os metadados técnicos:

- `_id_raw`: identificador técnico da ocorrência carregada;
- `_arquivo_origem`: identificação do arquivo de origem;
- `_carregado_em`: instante em que o registro foi incorporado à RAW.

A política de recarga é substitutiva e atômica: uma nova carga aprovada substitui integralmente a anterior. Falhas durante a operação provocam *rollback*, preservando o estado previamente publicado.

3.2 Camada CORE

O CORE materializa o modelo relacional aprovado. As transformações incluem:

- conversões explícitas de tipos;
- renomeações de atributos;
- aplicação das constraints físicas;
- derivação de prefixos de CEP;
- consolidação das duplicidades integrais de geolocalização;
- reconciliação dos volumes inseridos.

A ordem de dependências adotada é:

1. `core.prefixo_cep`;
2. `core.cliente`, `core.produto` e `core.vendedor`;
3. `core.pedido` e `core.geolocalizacao`;
4. `core.item_pedido`, `core.pagamento` e `core.avaliacao`.

3.3 Atomicidade do pipeline completo

O comando completo usa uma única conexão PostgreSQL para o fluxo CSV → RAW → CORE. O controle transacional é realizado pelo `psycopg`. Se qualquer arquivo, conversão, constraint ou reconciliação falhar, RAW e CORE retornam conjuntamente ao estado anterior.

4 Contrato de Ingestão RAW

A ingestão contempla exclusivamente os nove arquivos contratados. Arquivos adicionais exigem alteração explícita do contrato e dos artefatos físicos.

Arquivo	Linhas	Destino RAW
<code>olist_customers_dataset.csv</code>	99.441	<code>olist_customers</code>
<code>olist_geolocation_dataset.csv</code>	1.000.163	<code>olist_geolocation</code>
<code>olist_order_items_dataset.csv</code>	112.650	<code>olist_order_items</code>
<code>olist_order_payments_dataset.csv</code>	103.886	<code>olist_order_payments</code>
<code>olist_order_reviews_dataset.csv</code>	99.224	<code>olist_order_reviews</code>
<code>olist_orders_dataset.csv</code>	99.441	<code>olist_orders</code>
<code>olist_products_dataset.csv</code>	32.951	<code>olist_products</code>
<code>olist_sellers_dataset.csv</code>	3.095	<code>olist_sellers</code>
<code>product_category_name_translation.csv</code>	71	<code>product_category_name_translation</code>
Total	1.550.922	9 tabelas

A pré-validação confirma, antes de qualquer alteração no banco:

- presença dos nove arquivos obrigatórios;

- leitura UTF-8 ou UTF-8 com BOM;
- cabeçalho exato e na ordem contratada;
- quantidade correta de campos em todas as linhas;
- sintaxe CSV válida;
- quantidade de registros;
- tamanho e hash SHA-256 de cada fonte.

Campos vazios podem ser carregados como SQL NULL. Strings não vazias semelhantes a marcadores de ausência não são reinterpretadas. Duplicidades e inconsistências de negócio permanecem preservadas na RAW.

5 Mapeamento RAW para CORE

O mapeamento preserva identificadores de 32 caracteres como texto, prefixos de CEP como cinco caracteres, valores monetários em `numeric(12,2)` e datas em `timestamp without time zone`. Valores inválidos não são corrigidos ou descartados silenciosamente.

Origem	Destino CORE	Tratamento principal
olist_customers	cliente	Identificadores, CEP, cidade e UF
olist_orders	pedido	Status e eventos temporais
olist_order_items	item_pedido	Item, produto, vendedor, preço e frete
olist_products	produto	Categoria, descritores e medidas físicas
olist_sellers	vendedor	Identificador, CEP, cidade e UF
olist_order_payments	pagamento	Sequência, tipo, parcelas e valor
olist_order_reviews	avaliacao	PK composta, nota e datas
Clientes + vendedores + geolocalização	prefixo_cep	União distinta dos prefixos
olist_geolocation	geolocalizacao	Deduplicação exata e identity

Título e mensagem das avaliações e a tabela de tradução de categorias permanecem disponíveis na RAW porque não possuem destino no CORE aprovado.

6 Regras de Qualidade e Validação

A validação pós-carga é executada de forma independente do pipeline responsável pela transformação. Essa separação reduz o risco de uma implementação validar a si própria utilizando exatamente a mesma lógica que produziu o resultado.

O processo está organizado em quatro grupos de controles:

- **reconciliação**: comparação entre volumes esperados na RAW e volumes publicados no CORE;
- **integridade**: validação de unicidade, chaves primárias, UNIQUE e integridade referencial;
- **qualidade**: validação de nulabilidade, formatos, domínios, limites geográficos e valores numéricos;
- **transformação**: comparação do conteúdo da RAW com o conteúdo efetivamente publicado no CORE após as conversões previstas.

A execução validada contempla **87 regras automáticas**, distribuídas da seguinte forma:

Categoria	Regras	Resultado
Reconciliação RAW → CORE	9	Aprovadas
Integridade estrutural e referencial	19	Aprovadas
Validação dinâmica de NOT NULL	39	Aprovadas
Domínios e qualidade dos valores	11	Aprovadas
Correspondência das transformações	9	Aprovadas
Total	87	87 aprovadas

Entre os controles aplicados estão:

- ausência de duplicidades nas chaves primárias;
- unicidade de `pedido.id_cliente`;
- inexistência de registros órfãos nas nove relações de chave estrangeira;
- validação dinâmica de todas as 39 colunas NOT NULL existentes no CORE;
- validação dos formatos dos identificadores;

- prefixos de CEP com cinco dígitos e preservação de zeros à esquerda;
- validação dos domínios de UF, status de pedido e tipo de pagamento;
- valores monetários não negativos;
- sequenciais e quantidades compatíveis com seus domínios;
- notas de avaliação entre 1 e 5;
- dimensões físicas não negativas;
- latitude e longitude dentro dos limites geográficos válidos;
- ausência de cidades obrigatórias vazias;
- correspondência integral entre os valores transformados da RAW e os valores armazenados no CORE.

As verificações de transformação não se limitam à quantidade de registros. Para cada uma das nove entidades do CORE, o conjunto esperado é reconstruído a partir da RAW utilizando as conversões definidas no processo de carga e comparado ao conjunto efetivamente persistido.

Assim, uma tabela com o volume correto, mas contendo valores diferentes dos esperados, também reprova a validação.

A geolocalização constitui uma transformação deliberadamente não bijetiva. As 1.000.163 ocorrências preservadas na RAW resultam em 738.332 ocorrências distintas no CORE, correspondendo à consolidação de **261.831 duplicidades integrais**. A deduplicação considera conjuntamente prefixo de CEP, latitude, longitude, cidade e estado.

A entidade `prefixo_cep` é derivada da união dos prefixos presentes em clientes, vendedores e geolocalização, resultando em **19.177 prefixos distintos**.

O arquivo `product_category_name_translation.csv` permanece preservado na RAW, porém não participa da transformação atualmente aprovada para o CORE. O atributo `produto.nome_categoria` mantém o valor original `product_category_name`. Dessa forma, a ausência de uma tabela CORE correspondente ao arquivo de tradução não caracteriza perda acidental de dados.

7 Implementação

Artefato	Responsabilidade
models/physical/*.sql	Fonte da verdade da estrutura física, índices, desmontagem e validação do modelo
database/setup.py	Criação do database quando necessário e orquestração da preparação física
elt/raw_loader.py	Pré-validação, COPY, recarga substitutiva e reconciliação da RAW
elt/sql/load_core.sql	Transformações SQL executadas na ordem de dependências do modelo relacional
elt/core_loader.py	Execução das transformações e reconciliação inicial do CORE
elt/pipeline.py	Orquestração transacional do fluxo CSV → RAW → CORE
validation/elt_validation.py	Validação pós-carga independente de reconciliação, integridade, qualidade e correspondência das transformações
config/raw_load.toml	Contrato operacional das fontes RAW
config/core_load.toml	Configuração da transformação para o CORE
tests/test_database_setup.py	Testes automatizados da preparação do banco
tests/test_elt_validation.py	Testes automatizados das regras e do relatório de validação
tests/	Suíte automatizada dos componentes de preparação, carga, transformação e validação

8 Configuração e Execução

Credenciais não são versionadas. No ambiente local, a conexão pode ser armazenada em `.env`, ignorado pelo Git, enquanto `.env.example` documenta a variável necessária sem conter segredos.

```
DATABASE_URL=postgresql://usuario:senha@localhost:5432/ecommerce_analytics
```

A preparação inicial do banco é executada com:

```
uv run python -m database.setup
```

Se o database ainda não existir, ele é criado automaticamente antes da aplicação do modelo físico.

Para reconstruir a estrutura pertencente ao projeto em um database já existente:

```
uv run python -m database.setup --reset
```

Após a preparação física, a execução operacional do pipeline completo é:

```
uv run python -m elt.pipeline
```

A validação independente do resultado publicado é executada por:

```
uv run python -m validation.elt_validation
```

O fluxo completo e reproduzível é, portanto:

```
uv run python -m database.setup --reset
uv run python -m elt.pipeline
uv run python -m validation.elt_validation
```

A pré-validação das fontes sem conexão com o PostgreSQL continua disponível por:

```
uv run python -m elt.pipeline --validate-only
```

Cada execução completa do ELT gera um log JSON local em:

```
outputs/data-loading/logs/full_load_<run_id>.json
```

O registro contém perfis e hashes das fontes, reconciliações RAW e CORE, hash do SQL de transformação, horários, modo de execução, status e erro quando houver. A URL de conexão não é registrada.

A validação pós-carga gera separadamente:

```
outputs/data-loading/validation/elt_validation_<run_id>.json
```

Esse relatório registra o identificador da execução, database avaliado, horários, status geral, resumo das regras e resultado individual de cada controle.

9 Validação e Testes

A implementação foi submetida a validações automatizadas e testes de integração sobre banco PostgreSQL real.

Durante o desenvolvimento, o ELT foi validado em uma instância PostgreSQL 18.6 temporária e isolada. Duas execuções completas consecutivas demonstraram que a política de recarga não produz acumulação de registros. Uma falha deliberada ao final da transformação confirmou ainda que RAW e CORE participam da mesma unidade transacional e retornam conjuntamente ao estado anterior em caso de erro.

A automação de preparação do banco também foi validada em dois cenários: utilização de um database previamente existente e criação automática de um database inexistente. No segundo caso, o processo identificou a ausência pelo catálogo `pg_database`, criou o database, aplicou a estrutura física, criou os índices e concluiu sua validação.

A suíte específica de `database.setup` possui oito testes automatizados que cobrem:

- extração do nome do database a partir da URL;
- construção da URL administrativa apontando para `postgres`;
- não criação quando o database já existe;
- criação quando o database não existe;
- preparação normal sem remoção da estrutura;
- ordem dos scripts no modo `-reset`;
- ausência de remoção quando o database acaba de ser criado;
- falha controlada quando `DATABASE_URL` não está configurada.

A suíte específica de validação possui **16 testes automatizados**, incluindo cenários aprovados e reprovados para reconciliação, integridade, qualidade, nulabilidade e correspondência das transformações.

A suíte completa do projeto possui, no snapshot documentado, **44 testes automatizados**, todos aprovados.

Ela é executada por:

```
uv run python -m unittest discover -s tests -v
```

A validação final sobre a base persistente foi executada por:

```
uv run python -m validation.elt_validation
```

O resultado obtido foi:

APPROVED — 87 de 87 regras aprovadas

A execução completa da auditoria levou aproximadamente **8,5 segundos** no ambiente utilizado.

As nove verificações de correspondência RAW → CORE apresentaram zero divergências em:

- prefixo_cep;
- cliente;
- produto;
- vendedor;
- pedido;
- item_pedido;
- pagamento;
- avaliacao;
- geolocalizacao.

Portanto, além da igualdade dos volumes esperados, foi comprovado que os conteúdos publicados correspondem aos conjuntos derivados da RAW pelas transformações implementadas.

10 Resultado da Carga

Os volumes reconciliados no CORE para o snapshot validado são:

Tabela CORE	Registros reconciliados
prefixo_cep	19.177
cliente	99.441
produto	32.951
vendedor	3.095
pedido	99.441
item_pedido	112.650
pagamento	103.886
avaliacao	99.224
geolocalizacao	738.332
Total físico	1.308.197

10.1 Resultado da validação pós-carga

A reconciliação final não identificou divergências bloqueantes.

A camada CORE contém **1.308.197 registros físicos** distribuídos pelas nove entidades aprovadas. Esse total não deve ser comparado diretamente aos **1.550.922 registros da RAW** como se as duas camadas compartilhassem a mesma granularidade.

A principal redução ocorre em geolocalização:

1.000.163 ocorrências RAW → 738.332 ocorrências CORE

A diferença de **261.831 ocorrências** corresponde exclusivamente a duplicidades integrais consolidadas de forma determinística.

Além disso, `core.prefixo_cep` constitui uma entidade derivada com 19.177 registros, enquanto o arquivo de tradução de categorias, com 71 registros, permanece exclusivamente na RAW por não possuir destino definido no modelo CORE atual.

Todos esses comportamentos foram considerados explicitamente pelas regras de reconciliação e transformação e foram aprovados.

11 Rastreabilidade e Limites

A RAW mantém os valores originais e metadados técnicos que permitem relacionar cada ocorrência ao arquivo e à execução responsável por sua ingestão.

O pipeline registra perfis das fontes, quantidade de registros, hashes SHA-256, reconciliações, hash do SQL de transformação, horários e resultado da execução.

A validação pós-carga possui identificação independente e produz relatório JSON contendo o resultado individual das regras executadas. Dessa forma, uma reprovação pode ser associada à categoria e à regra que detectou a divergência.

A preparação do banco registra explicitamente o database de destino antes das operações estruturais. A URL de conexão e suas credenciais não são incluídas nos logs.

O modo `-reset` não remove o database PostgreSQL; apenas a estrutura pertencente ao projeto é reconstruída.

Embora a implementação atual identifique quantitativamente a regra violada, o registro detalhado de cada ocorrência inválida — incluindo, quando aplicável, arquivo de origem, `_id_raw`, coluna e valor original — permanece como possibilidade de evolução para fontes futuras que efetivamente apresentem violações bloqueantes.

No snapshot utilizado nesta entrega nenhuma das 87 regras executáveis apresentou violação.

Regras temporais e comportamentais que possam representar características ou anomalias legítimas da fonte, como diferenças incomuns entre eventos do ciclo do pedido, não foram transformadas automaticamente em critérios bloqueantes. Essas características pertencem à análise de qualidade semântica e poderão ser exploradas posteriormente sem modificar retrospectivamente os dados originais.

Agendamento, monitoramento operacional contínuo e objetos do schema `analytics` não fazem parte deste documento.

12 Conclusão

O processo de preparação, carga, transformação e validação dos dados está implementado de forma reproduzível e verificável.

O componente `database.setup` garante a existência do database e aplica o modelo físico versionado sem transferir essa responsabilidade ao ELT. O pipeline realiza a pré-validação das nove fontes contratadas, carrega 1.550.922 ocorrências na RAW e transforma os dados para as nove entidades do CORE dentro de uma única unidade transacional.

O resultado publicado contém 1.308.197 registros no CORE, incluindo 738.332 ocorrências únicas de geolocalização e 19.177 prefixos de CEP derivados.

A validação pós-carga é realizada por componente independente e verifica volu-

mes, integridade, nulabilidade, domínios e correspondência entre as transformações esperadas e os dados persistidos.

Na execução de referência, as **87 regras foram aprovadas sem divergências**. A suíte automatizada completa também foi concluída com **44 de 44 testes aprovados**.

Esses resultados demonstram que a camada CORE foi produzida de acordo com o modelo físico e com as regras de transformação definidas, sem perda silenciosa, acumulação indevida ou quebra conhecida de integridade.

A etapa de carga e transformação fica, portanto, concluída com evidência reproduzível e rastreável. O banco encontra-se apto para a construção das estruturas do schema `analytics` e para o desenvolvimento das análises destinadas ao consumo por SQL e ferramentas de Business Intelligence.

13 Referências

- OLIST. *Brazilian E-Commerce Public Dataset by Olist*. Kaggle. Disponível em: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>. Acesso em: 30 de agosto de 2026.
- POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation*. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 30 de agosto de 2026.
- PSYCOPG. *Psycopg 3 Documentation*. Disponível em: <https://www.psycopg.org/psycopg3/docs/>. Acesso em: 30 de agosto de 2026.
- PYTHON SOFTWARE FOUNDATION. *Python Documentation*. Disponível em: <https://docs.python.org/3/>. Acesso em: 30 de agosto de 2026.